Part A: Search Algorithms

Problem Formulation

The Eagle Delivery project simulates a robot navigating a campus grid to deliver packages between buildings. Each state represents the robot's position as a row, column coordinate. The initial state is the selected starting building, and the goal is to reach the destination building.

The robot can move up, down, left, or right. Valid movements change the position by one cell, while walls prevent movement. Each valid move has a path cost of 1. Because all moves have equal cost, Breadth-First Search (BFS) guarantees a shortest route when one exists.

Manhattan Distance and A*

I implemented the Manhattan distance heuristic which calculates the horizontal and vertical distance between the current position and the destination. This heuristic is acceptable because the robot cannot move diagonally, and walls may increase the required distance but cannot decrease it below the Manhattan distance.

I then implemented A* using a priority queue that selects positions based on f(n) = g(n) + h(n). The algorithm tracks the lowest known path cost and parent for each position, updates them when a cheaper route is found, and resolves equal-priority entries by insertion order.

BFS vs. A* Results

Map	Task	BFS length	A* length	BFS expanded	A* expanded
Baseline	S → W	18	18	67	22
Baseline	L → D	22	22	79	62
Baseline	H → P	16	16	41	16
Detour	S → W	26	26	70	51
Detour	L → D	22	22	65	60
Detour	H → P	16	16	41	16

Results

BFS and A* found routes of equal length in all six comparisons, confirming that both algorithms produced optimal paths. However, A* expanded fewer nodes in every experiment, demonstrating the benefit of using Manhattan distance to guide the search.

The detour increased the Student Center to Walton Hall route from 18 to 26 steps. In contrast, the Halle Library to Downing Hall and Pray-Harrold to Putnam Hall route lengths remained unchanged. This shows that blocked walkways do not affect every delivery route equally. The experiments also showed that a detour does not always increase the number of expanded nodes, since the Halle Library to Downing Hall search expanded fewer nodes on the detour map.

Part B: Q-Learning and Evaluation

Epsilon and Q-Learning

I implemented epsilon-greedy exploration and Q-learning to allow the delivery robot to learn navigation through experience. The Q-learning algorithm updates state-action values using the reward and maximum expected future reward, with no future reward included after a terminal transition.

Each delivery task was trained for 2,000 episodes using three independent seeds (0, 1, and 2). The parameters were $\alpha = 0.3$, $\gamma = 0.95$, $\varepsilon = 0.2$, and slip probability = 0.0. The learned policies were evaluated over 100 episodes per seed without further exploration or learning.

Baseline Results

Task	Success rate	Mean return	Mean steps
S → W	100%	22	18
L → D	100%	18	22
H → P	100%	24	16

All three delivery tasks achieved a 100% evaluation success rate across the three seeds. The learned routes matched the shortest paths identified by BFS and A*, indicating optimal delivery performance under deterministic conditions.

Training performance also improved with experience. For the H → P task, the average training success rate increased from 85% during episodes 1–100 to 100% from episodes 201–300 onward. These results demonstrate that Q-learning can learn efficient navigation policies from rewards and experience without using a search algorithm to plan the routes.

Part C: Custom Map and Exploration Experiment

Custom Map Design

I designed an 11 × 17 campus map containing all six buildings, blocked pathways, loops, and dead ends. The map includes a cycle near rows 7–8 and columns 6–7 and a dead-end corridor leading toward Pray-Harrold. BFS and A* both found a shortest route of 22 steps from the Student Center to Walton Hall. BFS expanded 60 nodes, compared with 34 for A*.

Research Question and Hypothesis

How does exploration affect learning and final performance in Q-learning?

I hypothesized that moderate exploration (ε = 0.2) would provide a good balance between discovering routes and exploiting learned information. I expected low exploration (ε = 0.05) to potentially limit discovery, while high exploration (ε = 0.5) could reduce training efficiency.

Experimental Method

I tested epsilon values of 0.05, 0.2, and 0.5 on the same custom map and S-W delivery task. Each setting used 2,000 training episodes with seeds 0, 1, and 2. I kept α = 0.3, γ = 0.95, slip probability = 0.0, and the 200-action episode limit constant. Each trained policy was evaluated over 100 episodes per seed without exploration or further learning.

Results

Epsilon	Final success	Mean steps	Last 500 training return
0.05	100%	22	16.79
0.20	100%	22	12.30
0.50	100%	22	−4.48

All three settings reached a 100% training success rate by episodes 101 to 200 and maintained it in subsequent blocks. Final evaluation performance was identical, with each setting achieving optimal 22-step deliveries and a mean return of 18.

However, training rewards differed significantly. Low exploration produced the highest returns, while high exploration resulted in lower returns because the agent continued selecting unnecessary random actions.

Discussion and Conclusion

My results kind of support the hypothesis. Higher exploration reduced training efficiency. However, epsilon = 0.05 performed better during training than epsilon = 0.2, even though my prediction was that moderate exploration would provide the best balance.

All three settings learned an equally effective policy. This suggests that for the custom map and 2,000 episode training budget, exploration rate affected training rewards more than final delivery performance. The results may differ with more complex maps, shorter training periods, or uncertain movement.